

Sabre 2026 robot code walkthrough

How the dumper branch of sabre-code-2026 fits together, with diagrams of the architecture, the control loop, pose estimation, and shooting. Generated 2026-09-19 from commit 2b47477.

Overview

This is a WPILib command-based FRC robot written in Java, with AdvantageKit logging and Choreo trajectories. On the dumper branch the turret and adjustable hood were removed, so the shooter is a fixed-angle dumper: spin two flywheels to a speed, then push balls into them with a kicker and a floor roller. The codebase is about 17,000 lines, but most of that is simulation and math utilities. The parts that run on the competition robot are four subsystems, a dozen commands, and the auto routines.

1. The layers

Everything follows the same top-down pattern. A command decides *what* to do, a subsystem exposes *operations*, and an IO interface hides *which hardware* performs them. The Physical classes are the only place CTRE TalonFX or Limelight calls live, so a subsystem can be swapped to a simulated or replay implementation without touching any command. That swap is what the RobotType switch in Robot.setupSubsystems() does; only the COMP_ROBOT case currently builds all four subsystems.

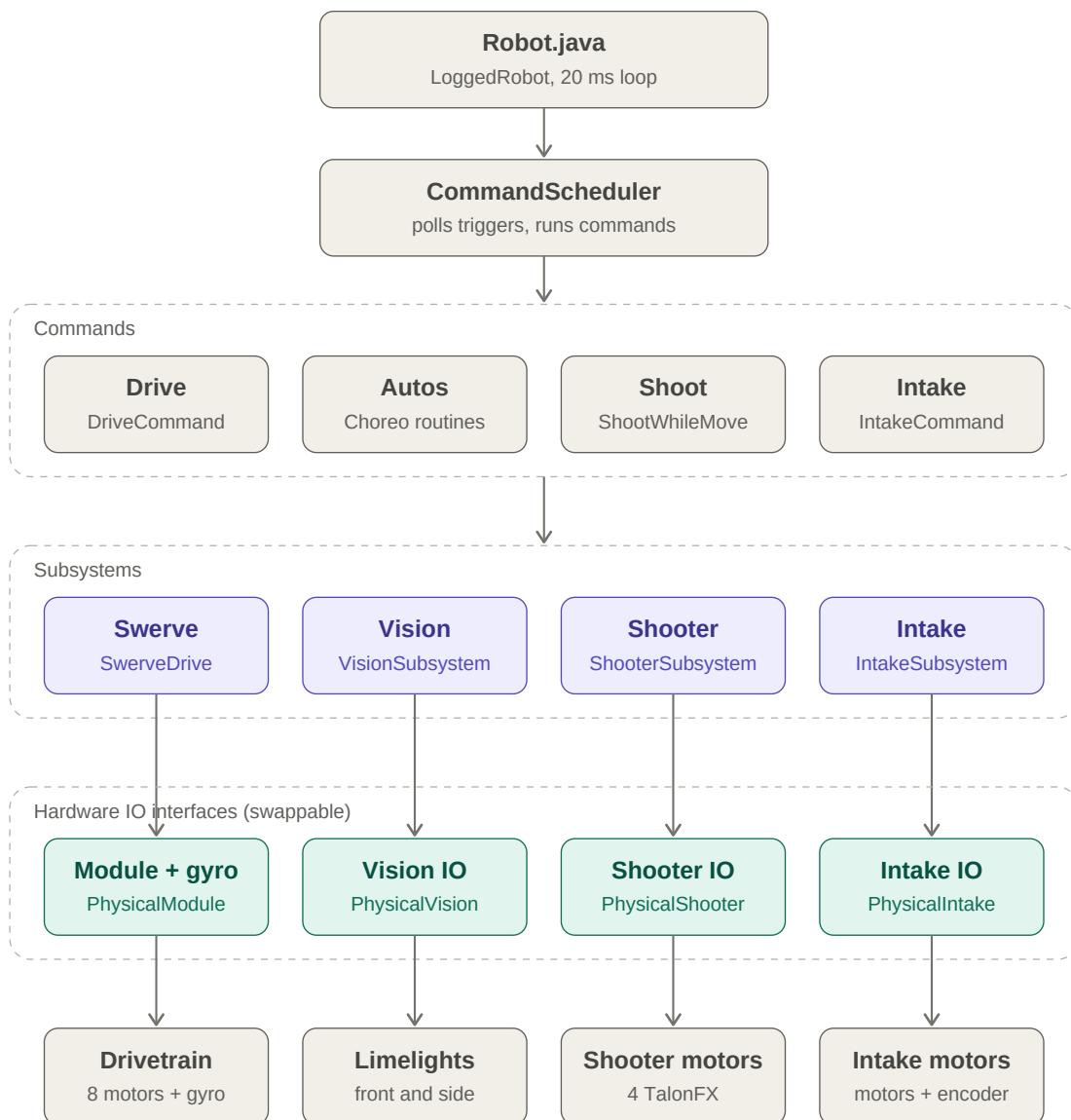


Figure 1. Each arrow is a plain Java method call. The Drive and Autos commands also read the vision subsystem, not only the column beneath them.

The Physical implementation is handed to each subsystem through its constructor. For example `new ShooterSubsystem(new PhysicalShooter())` on the competition robot, versus `new ShooterSubsystem(new ShooterInterface() {})` for a replay where every IO method is a no-op default.

2. What happens every 20 milliseconds

Nothing in this codebase runs on its own timer except the Limelight polling threads. Everything else is driven by one loop in `robotPeriodic()`, which raises the thread to real-time priority, runs the command scheduler once, and lowers priority again.

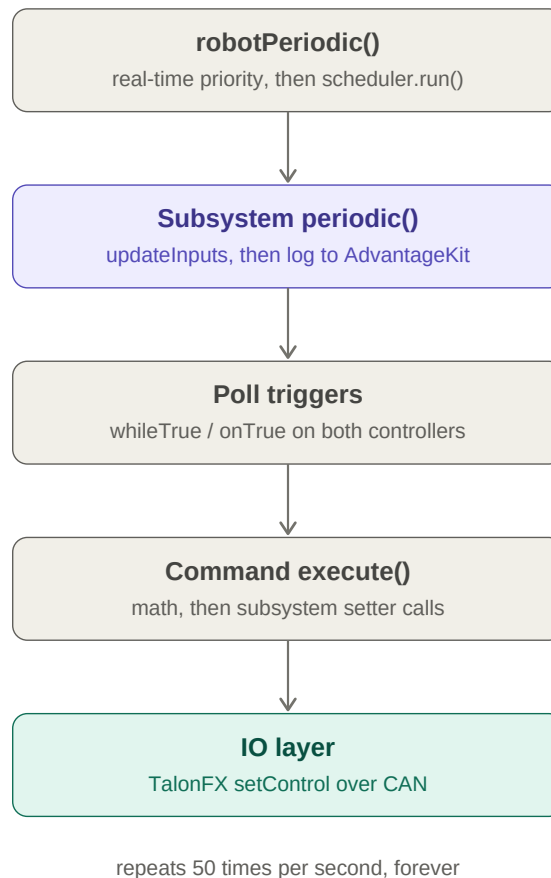


Figure 2. One scheduler tick. Steps two through five happen inside `CommandScheduler.run()`.

Two details matter here. First, every subsystem's `periodic()` refreshes its sensor signals into an `Inputs` object and hands it to the AdvantageKit logger, so every value the code sees is recorded and can be replayed later. Second, the bindings are almost all `whileTrue`, so a command like `IntakeCommand` runs its `execute()` fifty times a second for as long as the operator holds the trigger, and its `end()` stops the motors on release.

3. Where the robot thinks it is

The drivetrain keeps a `SwerveDrivePoseEstimator`. Wheel odometry from the four modules plus the Pigeon gyro is added on every drive tick. Each Limelight's MegaTag pose is fused in with a confidence that depends on how many AprilTags it sees, how far away they are, and how fast the robot is moving; more tags, closer tags, and a slower robot all lower the standard deviation and so raise the trust in that measurement. This logic lives in `DriveCommandBase`, which both the teleop `DriveCommand` and the Choreo `FollowSwerveSampleCommand` extend.

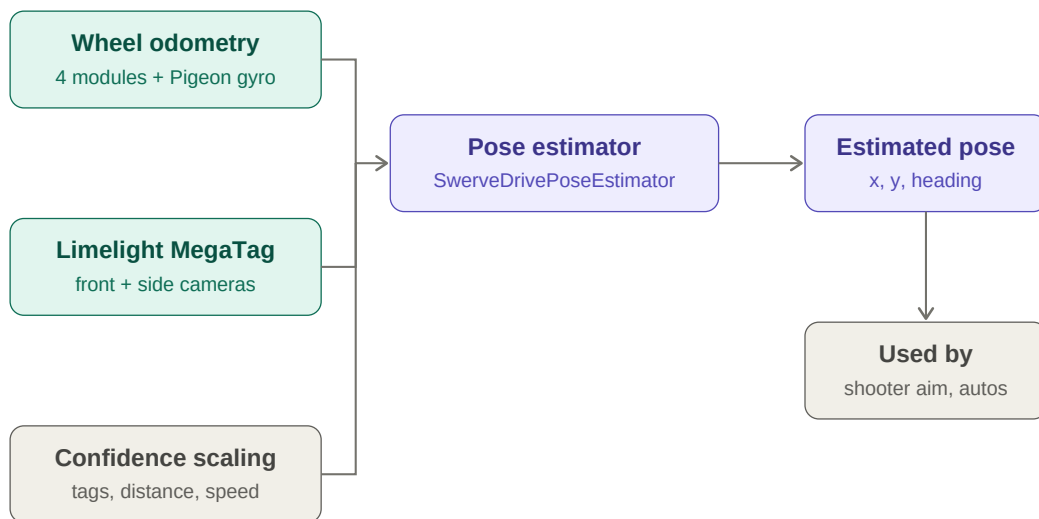


Figure 3. Odometry and vision both feed one estimator. The confidence scaling is applied to the vision measurement before it is added.

The driver can also force the estimate. D-pad left snaps the pose to the last MegaTag1 pose seen, and D-pad right resets only the heading to the alliance-relative zero.

4. Shooting on the dumper branch

With the turret gone, shooting is: figure out how far away the hub is, spin the flywheels, and run the feed. The commands still do the shoot-on-the-move math. They shift the hub target backwards by the robot's field-relative velocity multiplied by an estimated ball flight time, and iterate twenty times because the flight time depends on the very distance it changes. `ShootWhileMove` aims at the hub center; `DumperShootCommand` aims at a fixed x coordinate and picks a y of 1.6 m or 6.3 m depending on which half of the field the robot is on.

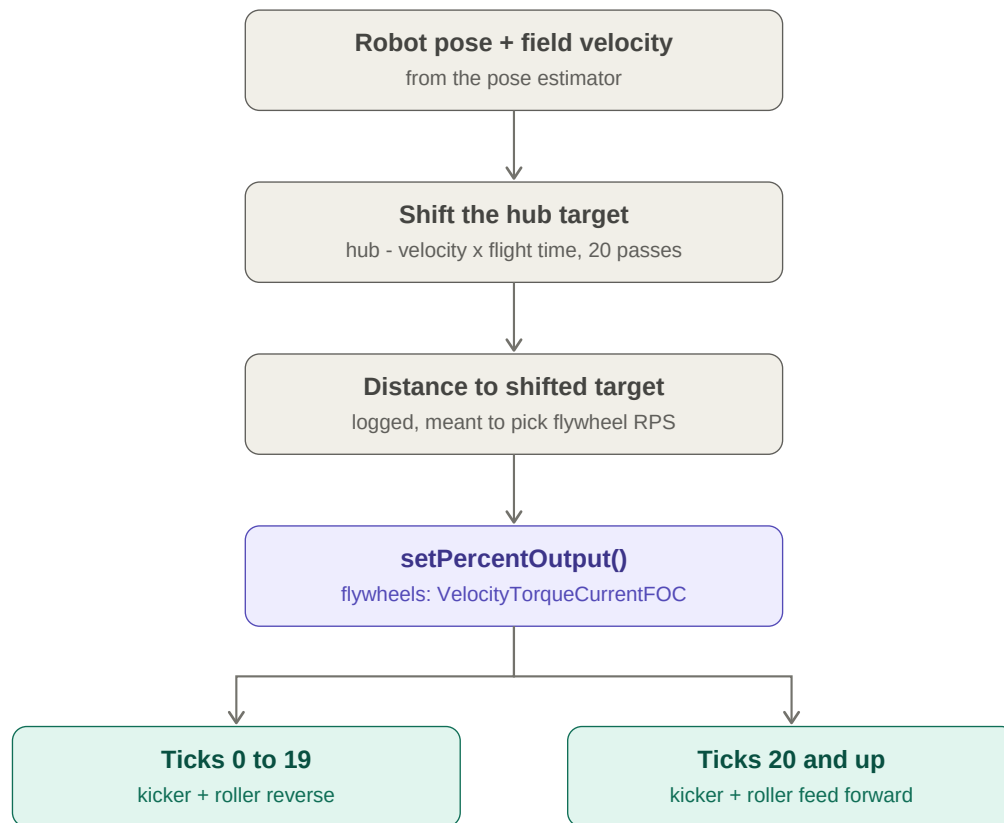


Figure 4. Shot flow. The two-phase feed at the bottom is the whole dumper trick.

The first twenty ticks pull balls back so nothing is jammed against the flywheel while it spins up. After that the kicker and front roller push forward continuously. There is a more elaborate pause-and-resume roller state machine below those two branches in `PhysicalShooter.setPercentOutput`, but both branches return early so it is currently unreachable.

5. Controls

Controller	Input	Command
Driver	Left stick, right stick X	DriveCommand (default). Field relative unless right bumper is held.
Driver	Right trigger	ShootWhileMove, and slow-mode driving at the same time.
Driver	B	DumperShootCommand.
Driver	A	DefenseCommand (intake all the way up).
Driver	Left trigger	ReverseKickerAndRollers.
Driver	Left bumper	Trench align: PID the heading to 0 or 180 degrees.
Driver	D-pad left / right	Reset pose to vision / reset heading only.
Operator	Right trigger	IntakeCommand (pivot down, run rollers once deployed).
Operator	Left trigger	OuttakeCommand (intake and shooter roller reversed).
Operator	Y / A / X / B	Pivot up / pivot down / bounce lower / nudge up open-loop.
Operator	D-pad up	Zero the pivot CANcoder.

6. Autonomous routines

Autos registers seven Choreo routines with a dashboard chooser and rebuilds the selected one whenever the selection or alliance changes while the robot is disabled. Every routine has the same shape:

- 1 Reset odometry to the trajectory start, then drive the first sweep while the intake pivots up briefly and then intakes.
- 2 Stop for a tenth of a second with DriveNoVision.
- 3 Run ShootWhileMove for five seconds while the intake bounces to unjam balls.
- 4 Drive a second sweep with the intake running, stop, and shoot again. The two depot autos skip the second sweep and shoot for ten seconds instead.

Trajectory following is a feedforward from the Choreo sample plus three PID controllers on x, y, and heading, computed in `SwerveDrive.followSwerveSample`. The result is negated before being sent to `drive()`, which suggests the path coordinates and the robot's drive convention are mirrored relative to each other.

7. Things to look at

These were noticed while reading the code and were not changed.

- **Null pointer on every shot.** Both shooting commands read a `turretPose` field that is never assigned on this branch. On main it was computed from the robot pose plus a turret offset; the commit that removed the turret deleted that line but kept the reads. The first `execute()` of `ShootWhileMove` or `DumperShootCommand` will throw, which will also end every auto routine at its shoot step. The fix is one line at the top of each `execute()`: `turretPose = robotPose.getTranslation();` See `ShootWhileMove.java:91` and `DumperShootCommand.java:89`.
- **Distance does not set the flywheel speed.** All of the shift-the-target math ends in a call to `setPercentOutput(distance, ...)`, but that method ignores the distance and reads the tunable `Shooter/RPS` from the dashboard instead. The distance lookup table is commented out at `PhysicalShooter.java:157`. That may be intentional for a fixed-angle dumper, but the third box in Figure 4 is currently logging only.
- **Aiming check is bypassed.** In `ShootWhileMove`, the `isAimingProperly` branch is immediately followed by an unconditional `setPercentOutput` call, so the shooter always runs regardless of the flag.